

BARCODE Volumetric Metrics

Formal specification of the metrics computed by `analysis/volumetric/`

This document is the formal counterpart to the published two-dimensional reference, *BARCODE 2.0 Metrics*, and follows its conventions: numbered branches, one entry per metric, numbered display equations, and a statement of units and failure conditions for every quantity. Equation numbering here is section-scoped rather than continuous, so that inserting an equation does not renumber the document.

The 2D reference defines **25** parameters across three branches. This pipeline extends that work to volumes rather than replacing it: it emits up to **63** across ten families, of which all 25 carry over — 17 under identical names, 8 renamed *Area* to *Volume* — while the remaining 38 are measurements a volume affords that a plane does not. Restating a metric for a volume sometimes changes what it measures; where the two are *not* numerically comparable, the entry says so. The 2D branch is unchanged and remains the reference for 2D data.

NEW CAPABILITY: BARCODE CAN NOW BE GIVEN A SEGMENTATION

Published BARCODE defines what counts as material by thresholding intensity at $\bar{B} \times (1 + \text{offset})$ — a self-contained rule needing nothing but the image. **This pipeline adds a second route: an external segmentation mask, which when it resolves replaces the threshold.** Objects are then whatever the segmenter says they are. Both routes are supported; the mask is an addition rather than a correction, and it is the largest behavioral difference from the 2D reference.

Two metric families cannot exist without one: **In-Mask Intensity** (§4.29–4.37) and **Packing Topology** (§4.38–4.44) — the latter additionally requiring integer instance labels. Two more *change*: **Surface Mesh** (§4.1–4.11) and **Curvature** (§4.12–4.18) are still produced without a mask, but they then mesh the *binarized volume* — a threshold surface, not the object. It also changes families that do exist without one: the binarization branch (§1) then measures the segmented object, and flow and the intensity histogram may be restricted to in-mask voxels (`flow_use_mask`, on by default; `intensity_use_mask`, off).

Formats: `.tif / .tiff`, `.png / .bmp`, `.npy`, `.npz` — including Cellpose `_seg.npy` bundles. **Instance labels are preserved** rather than collapsed to a boolean, which is what makes a confluent field measurable at all. Pairing is by regex and path template, so a batch resolves automatically, and every validation **raises** rather than degrading (§P.3).

Contents

Front matter

§0 Notation and common reductions

§P Measurement preconditions

P.3 Segmentation input (**new**)

1. Image Binarization (3D)

1.1 Connectivity $\mathcal{C}$

1.2 Maximum island volume I

1.3 Maximum void volume V

1.4 Maximum island volume change ΔI

1.5 Maximum void volume change ΔV

1.6–1.7 Initial max & 2nd max island volumes $I_{0,1}, I_{0,2}$

1.8 Mean island anisotropy A_I

1.9 Mean island volume $\bar{I}$

- 1.10 Total island volume I_{tot}
- 1.11 Mean island separation D_I
- 1.12 Structural correlation length ξ_I

2. Intensity Distribution

- 2.1 Maximum kurtosis K
- 2.2 Maximum median skewness S_1
- 2.3 Maximum mode skewness S_2
- 2.4–2.6 Change metrics $\Delta K, \Delta S_1, \Delta S_2$
- 2.7 Total intensity Φ
- 2.8 Mean intensity $\bar{\Phi}$
- 2.9 Intensity SD σ_Φ
- 2.10 Intensity density ρ_Φ

3. Optical Flow (3D)

- 3.1 The solver
- 3.2 The validity mask
- 3.3 Speed v
- 3.4 Speed change Δv
- 3.5 Mean flow direction θ
- 3.6 Directional spread σ_θ
- 3.7 Velocity correlation length ξ_v

- 3.8 Divergence $\nabla \cdot \mathbf{v}$

- 3.9 Curl Ω

4. Volumetric Branch

Runs without a segmentation

- 4.1–4.11 Surface mesh
- 4.12–4.18 Curvature
- 4.19–4.22 Object size distribution
- 4.23–4.28 Depth profile

Requires a segmentation

- 4.29–4.37 In-mask intensity
- 4.38–4.44 Packing topology

Per object

- 4.45 Object rows

Recording what was analyzed

- 4.46 Range provenance

Reading the outputs

Column index (all modes)

5. Warnings and flags

6. Known limitations

§0 Notation and common reductions

The analyzed unit is a volume. Let $B(x, y, z, i)$ be the intensity of the voxel at position (x, y, z) in the i^{th} timepoint, with $0 \leq x < n_x, 0 \leq y < n_y, 0 \leq z < n_z$, so that the analyzed field contains

$$N_v = n_x n_y n_z \quad (0.1)$$

voxels. Physical voxel dimensions are $(\Delta x, \Delta y, \Delta z)$ in micrometers, giving a voxel volume

$$v_0 = \Delta x \Delta y \Delta z \quad [\mu\text{m}^3]$$

All of N_v , v_0 and the axis extents refer to the *analyzed* grid — that is, after any isotropic resampling and cropping (§P.4), not the acquired grid.

0.1 Timepoint selection

Not every timepoint is analyzed. Given N_T timepoints and a stride k (`frame_step`, default **10**), the analyzed index set is

$$\mathcal{F} = \{0, k, 2k, \dots\} \cup \{N_T - 1\}, \quad N_f = |\mathcal{F}| \quad (0.2)$$

where k is first reduced by $k \leftarrow \max(\lfloor k/5 \rfloor, 1)$ while $k \geq N_T$, and $\mathcal{F} = \{0\}$ when $N_T \leq 1$. The final index is always included. The optical flow branch does not use $\mathcal{F}$ in this way; see §3.2.

0.2 The three reductions

Every metric produces one value per analyzed timepoint and is then reduced by one of three operators. Writing $\{q_i\}_{i \in \mathcal{F}}$ for the per-timepoint values:

Finite-only mean. The default reduction, used unless stated otherwise:

$$\langle q \rangle = \frac{1}{|\mathcal{G}|} \sum_{i \in \mathcal{G}} q_i, \quad \mathcal{G} = \{i \in \mathcal{F} : q_i \text{ finite}\} \quad (0.3)$$

with $\langle q \rangle = \text{NaN}$ when $\mathcal{G} = \emptyset$. A timepoint that failed therefore leaves the average rather than poisoning it, which also means a metric may be finite while most timepoints failed.

Top-decile mean. Used by the four *maximum* metrics. Let $q_{(1)} \geq q_{(2)} \geq \dots \geq q_{(m)}$ be the m finite per-frame values in decreasing order (non-finite frames are dropped first) and $n_{10} = \lceil 0.1 m \rceil$:

$$\text{top}_{10}(q) = \frac{1}{n_{10}} \sum_{j=1}^{n_{10}} q_{(j)} \quad (0.4)$$

The decile fraction **0.1** is fixed in the source and is not a user parameter. The decile is taken over the finite count m , not the analyzed-frame count N_f .

Head–tail comparison. Used by every *change* metric. With $f = \text{percentage_frames_evaluated}$ (default **0.05**) and

$$n_e = \max(\lceil f N_f \rceil, 1) \quad (0.5)$$

the head and tail means are $\mu_{\text{head}} = \langle q_i \rangle_{i \in \text{first } n_e}$ and $\mu_{\text{tail}} = \langle q_i \rangle_{i \in \text{last } n_e}$. The binarization branch forms a *ratio* and the intensity and flow branches a *difference*:

$$\Delta^{\text{ratio}} q = \frac{\mu_{\text{tail}}}{\mu_{\text{head}}}, \quad \Delta^{\text{diff}} q = \mu_{\text{tail}} - \mu_{\text{head}} \quad (0.6)$$

Single-timepoint rule. When $N_f < 2$ every change metric returns **NaN**, not **0** or **1**. A single volume is the ordinary case in this pipeline, so this is encountered constantly. The 2D branch instead compares a frame with itself and returns **0** (differences) or **1** (ratios).

0.3 Size reporting: fraction and quantity

Every size metric is reported twice. For a raw voxel count s ,

$$s^{\text{frac}} = \frac{s}{N_v} \quad [\text{dimensionless}, 0-1], \quad s^{\mathcal{Q}} = s v_0 \quad [\mu\text{m}^3] \quad (0.7)$$

The fractional column carries the unit label `fraction of FOV` but holds a fraction in $[0, 1]$, following the 2D convention. The two forms are *not* interchangeable: s^{frac} depends on the crop (SP.4) and $s^{\mathcal{Q}}$ does not, so only $s^{\mathcal{Q}}$ is comparable between runs of differing crop. The quantity columns are written to a separate `... (physical).csv` beside the normalized one (see *Reading the outputs*).

§P Measurement preconditions

Every quantity in this document is conditioned on the following. A reader who does not know which of these applied cannot interpret any reported number.

P.1 Axes are declared, never inferred

The reader takes the file's declared axis order and raises on anything it cannot interpret: the placeholder axes **I**, **Q**, **S**; any axis outside `TZCYX`; a missing **Y** or **X**; and a missing **Z**. It never falls back on shape heuristics. An `axes_override` may *tell* it the true order — one letter per data dimension — which replaces the declared order outright.

P.2 Voxel size and frame interval

Precedence is explicit setting $\rightarrow$ file metadata $\rightarrow$ warn and default to **1.0 μm** . The **z** step falls back to the ImageJ `spacing` tag and the **xy** step to $\mu\text{m}_{\text{per unit}}/\text{px}_{\text{per unit}}$ from the resolution tag.

Resolution tags are read through a unit name, not assumed to be microns. The unit comes from ImageJ's `unit` field, else the TIFF `ResolutionUnit`. A tag naming an unrecognized unit, or one declaring the image explicitly uncalibrated (`pixel`, `px`, `none`, `a.u.`), is discarded — it warns and falls back to **1.0 μm** rather than reading a pixel count as microns. Only a wholly absent unit takes the older "assume microns" path.

The frame interval must be set by hand. It defaults to **0**, and the fallback is narrower than it looks: the file's value is used **only** where the reader positively trusts the file's timing. An ND2 acquisition time loop qualifies; an ImageJ `finterval` on a per-timepoint TIFF export does **not** — there it frequently describes the **z** acquisition or is left at **1**, and nothing in the file distinguishes the two — so the reader returns **1.0** outright and says so. Most importantly, a **grouped time-lapse never uses the file's timing at all**: assembling per-file volumes into a series marks the timing untrusted, so unless you set the interval the whole series runs at **1.0** s. Since $v \propto 1/\Delta t$ exactly and nothing else changes, the error is invisible in the output.

P.3 Segmentation replaces thresholding

New in the volumetric pipeline; published **BARCODE** has no segmentation input. Set `segmentation_enabled` and point the resolver at the masks. Supported formats are `.tif/.tiff`, `.png/.bmp`, `.npy` and `.npz`, dispatched on the suffix, where a `.npy` may be a bare label array or a Cellpose `_seg.npy` bundle unwrapped through its `masks/mask/labels` key. A second independent mask may accompany the first, so a nucleus and its cell are both available for one image.

When a mask resolves it *is* the binarization: the threshold, minimum island size and inversion are bypassed for the structural branch, while intensity and autocorrelation continue to use raw voxels. Validation raises rather than degrading — **XY** dimensions must match exactly, and the **z** extents must agree to within **1.0 μm** :

$$|n_z \Delta z - n_z^{\text{mask}} \Delta z^{\text{mask}}| \leq 1.0 \mu\text{m} \quad (\text{P.1})$$

A mask carrying more than one distinct positive value keeps its integer labels; collapsing it to a boolean would re-derive objects by connectivity and merge every touching instance.

P.4 Isotropic resampling and the crop box

Resampling happens with or without a segmentation. With `make_isotropic` on (the default) and no mask, the image is still resampled to isotropic-at-**xy** from its own acquired spacing — so a maskless run **does** change slice count (54 $\rightarrow$ **245** on the Jurkat geometry), which moves the maximal-area slice index, every fraction denominator, and the spacing intensity density divides by. Only `make_isotropic = False` leaves the acquired grid untouched. What a mask adds is the *cropping* option below, and the mask-gated families.

With a mask and `make_isotropic` enabled: the image is resampled onto the mask grid (linear interpolation), then both onto an isotropic grid at the **xy** spacing — images linearly, masks by nearest neighbor, since any interpolating kernel would average neighboring instance labels into new and meaningless ones. Where the mask grid overhangs the image the last acquired plane is repeated rather than zero-filled: zero-filling would inject background voxels into a region the mask still declares foreground, and those voxels reach the mean-relative threshold, the histogram and every fraction denominator.

It does **not** crop. `crop_to_mask` defaults to **False**, so N_v in (0.1) is the acquired field of view.

Why cropping is off. Every fractional size in (0.7) is a fraction of *the analyzed box*. Cropping to the mask gives each file its own denominator, so the same object in a tighter box reports a larger fraction and a genuine size trend cannot be told apart from the box tracking the object. Turning it on restores the old behavior and crops a series to the *union* of the per-frame boxes, never each to its own. The μm^3 quantity columns are unaffected either way — prefer them when in doubt. The intended way to analyze less than the full field is an explicit z range.

P.5 Range restriction

The analyzed ranges satisfy $z \in [z_0, z_1]$ and $i \in [t_0, t_1]$, **inclusive of both ends**, with **end = 0** meaning "to the end" and negative values counting back from the end (**-1** being the last). So $z_0 = 12$, $z_1 = 46$ analyzes slices 12 through 46 — 35 slices — and $z_0 = z_1$ is a single slice rather than an empty range. An empty or reversed range raises. Any restriction sets flag digit 5. Because "slice 46" is ambiguous between the acquired and isotropic grids (54 versus **245** planes on **0.3/0.065 μm** data), the z range carries an explicit unit: acquired index, isotropic index, or micrometers of depth. In `xyzt` the acquired range is mapped onto a finer mask grid by **round($i(m_z - 1)/(n_z - 1)$)**, so a restricted image and its full-depth mask stay aligned over the sub-range rather than the mask being planted at $z = 0$.

`t_range_units="seconds"` converts through the **configured** frame interval, falling back to the file's tag only when the file stated timing and otherwise raising rather than assuming **1 s**. Without a stated interval a request in seconds cannot be resolved at all — "the first 600 seconds" would otherwise select 600 timepoints from a 60-second series.

1 Image Binarization (3D) Branch

The structural branch. Twelve columns, measuring volumes with 26-connectivity where the 2D branch measures areas with 8-connectivity. Throughout this section a *frame* means one timepoint — an entire $\mathbf{Z}$ -stack — not a $\mathbf{z}$ -slice.

1.0 Producing the binary volume

Absent a segmentation, the threshold is the volume mean scaled by a user offset (default **0.1**), computed over all voxels of the i^{th} volume:

$$\bar{B}(i) = \frac{1}{N_v} \sum_{x,y,z} B(x,y,z,i) \quad (1.1)$$

$$b(x,y,z,i) = \begin{cases} 1, & \text{if } B(x,y,z,i) \geq \bar{B}(i) \times (1 + \text{offset}) \\ 0, & \text{if } B(x,y,z,i) < \bar{B}(i) \times (1 + \text{offset}) \end{cases} \quad (1.2)$$

Connected objects and holes smaller than $s_{\min} + 1$ voxels are then removed under 26-connectivity, where $s_{\min}$ is `minimum_island_size` (default **1**, so isolated single voxels are discarded). If inversion is selected, $b \leftarrow 1 - b$.

A constant volume short-circuits to an empty binary. A blank or flat-saturated acquisition has zero range, and (1.1)–(1.2) would then give a threshold of **0** and mark *every* voxel as foreground — one island filling the field, $C = 1$, and a CSV row indistinguishable from a genuinely percolating sample.

There is no spatial binning in 3D: the 2D branch's p_b has no analogue here, and (0.1) is the unbinned voxel count.

Let $\{\mathcal{R}_1(i), \dots, \mathcal{R}_n(i)\}$ be the 26-connected foreground components of frame i , ordered so that $|\mathcal{R}_1| \geq |\mathcal{R}_2| \geq \dots$, where $|\cdot|$ denotes voxel count. When an instance mask supplies labels, that partition is used directly instead of connected-component labeling.

1.1 Connectivity C

COLUMN Connectivity · **UNIT** fraction of frames, valued **0–1**

The fraction of analyzed frames in which a single component percolates the volume. Labeling under the full $3 \times 3 \times 3$ structuring element, let $\mathcal{L}_0^a$ and $\mathcal{L}_1^a$ be the sets of non-zero labels appearing on the two opposing faces normal to axis a . Then

$$C_i = \begin{cases} 1, & \text{if } \exists a \in \{x,y,z\} : \mathcal{L}_0^a \cap \mathcal{L}_1^a \neq \emptyset \\ 0, & \text{otherwise} \end{cases} \quad C = \frac{1}{N_f} \sum_{i \in \mathcal{F}} C_i \quad (1.3)$$

Versus 2D. The 2D metric tests two axes under 8-connectivity; this tests three under 26-connectivity, so an object spanning depth but not the lateral field now scores **1**. Never **NaN**.

1.2 Maximum island volume I

COLUMN Maximum Island Volume · **UNIT** fraction of FOV (0–1); quantity form $\mu\mathbf{m}^3$

The size of the largest component, reduced by the top-decile mean (0.4) and normalized by (0.7):

$$I = \frac{\text{top}_{10}(|\mathcal{R}_1(i)|)}{N_v}, \quad I^Q = \text{top}_{10}(|\mathcal{R}_1(i)|) v_0 \quad (1.4)$$

NaN when every analyzed frame is empty.

Versus 2D. The 2D "Area Quantity" multiplies a pixel count by the micron-per-pixel ratio *once*, so it carries $\mu\mathbf{m}\cdot\mathbf{px}$; equation (0.7) multiplies by the full voxel volume and is in $\mu\mathbf{m}^3$. The two cannot be pooled or compared, whatever the shared column name suggests.

1.3 Maximum void volume V

COLUMN Maximum Void Volume · **UNIT** fraction of FOV (0–1); quantity form $\mu\mathbf{m}^3$

As (1.4) but over the 26-connected components of the complement $\mathbf{1} - b$.

EDGE CASE — WHERE THE TWO BRANCHES DIFFER

If the foreground fills the volume so that no background component exists, this branch reports **0**, reading the empty set as "no void exists"; the 2D branch returns the whole frame. Both are defensible readings, but they sit at opposite ends of the range, and the choice feeds ΔV as well — so the 2D and 3D columns are not comparable on a filled field.

1.4 Maximum island volume change ΔI

COLUMN Maximum Island Volume Change · **UNIT** ratio to initial · no quantity form

The ratio form of (0.6), evaluated on raw voxel counts so that the result is scale-free:

$$\Delta I = \frac{\langle |\mathcal{R}_1(i)| \rangle_{\text{last } n_e}}{\langle |\mathcal{R}_1(i)| \rangle_{\text{first } n_e}} \quad (1.5)$$

NaN when $N_f < 2$. Values above **1** indicate growth.

1.5 Maximum void volume change ΔV

COLUMN Maximum Void Volume Change · **UNIT** ratio to initial

Equation (1.5) applied to the void series, with the same single-frame rule.

1.6–1.7 Initial maximum and 2nd maximum island volumes $I_{0,1}$, $I_{0,2}$

COLUMNS Initial Maximum Island Volume, Initial 2nd Maximum Island Volume · **UNIT** fraction of FOV (0–1); quantity forms $\mu\mathbf{m}^3$

$$I_{0,1} = \frac{1}{N_v} \langle |\mathcal{R}_1(i)| \rangle_{\text{first } n_e}, \quad I_{0,2} = \frac{1}{N_v} \langle |\mathcal{R}_2(i)| \rangle_{\text{first } n_e} \quad (1.6)$$

with the convention $|\mathcal{R}_2(i)| = 0$ when frame i contains a single object. Hence $I_{0,2} = 0$ — a meaningful statement, not a missing value — for a single-nucleus run. Unlike the change metrics, these are *not* **NaN** for a single timepoint.

1.8 Mean island anisotropy A_I

COLUMN Mean Island Anisotropy · **UNIT** dimensionless, ≥ 1

Let $\lambda_1 \geq \lambda_2 \geq \lambda_3$ be the eigenvalues of the inertia tensor of a component. The equivalent-ellipsoid major and minor axis lengths are

$$a = \sqrt{10 \max(\lambda_1 + \lambda_2 - \lambda_3, 0)}, \quad c = \sqrt{10 \max(-\lambda_1 + \lambda_2 + \lambda_3, 0)} \quad (1.7)$$

$$A_i^k = \begin{cases} a/c, & c > 10^{-6} \\ \text{NaN}, & \text{otherwise} \end{cases} \quad A_I = \left\langle \langle A_i^k \rangle_k \right\rangle_i \quad (1.8)$$

Measured in physical coordinates, so it describes the object, not the sampling grid. The inertia tensor is built with the physical voxel spacing, so a physically spherical object reads ≈ 1 on any grid — where an index-space tensor would report the grid's own aspect ratio (e.g. 4.6 on the z 0.3 / xy 0.065 $\mu\mathbf{m}$ Jurkat geometry). On an isotropic grid the distinction vanishes: uniform scaling multiplies every eigenvalue by s^2 , leaving the ratio unchanged.

The clamp in (1.7) is not cosmetic. The library's axis-length properties raise a domain error on flat or thin 3D regions; on one real nucleus **135 of 167** components were degenerate. Degenerate components are excluded from the mean, not zeroed. Consequently A_I is a 3D ellipsoid ratio and is *not* comparable with the 2D ellipse ratio.

1.9 Mean island volume $\bar{I}$

COLUMN Mean Island Volume · **UNIT** fraction of FOV (0–1); quantity form $\mu\mathbf{m}^3$

$$\bar{I} = \frac{1}{N_v} \left\langle \frac{1}{n_i} \sum_{k=1}^{n_i} |\mathcal{R}_k(i)| \right\rangle_i \quad (1.9)$$

1.10 Total island volume I_{tot}

COLUMN Total Island Volume · **UNIT** fraction of FOV (0–1); quantity form $\mu\mathbf{m}^3$

The foreground volume fraction of the analyzed field:

$$I_{tot} = \frac{1}{N_v} \left\langle \sum_{k=1}^{n_i} |\mathcal{R}_k(i)| \right\rangle_i \quad (1.10)$$

With a segmentation, I_{tot}^Q is effectively the segmented volume in $\mu\mathbf{m}^3$ — the branch's most direct measure of object volume, and the natural cross-check against the mesh volume of §4.2.

1.11 Mean island separation D_I

COLUMN Mean Island Separation · **UNIT** $\mu\mathbf{m}$ · no quantity form

Component centroids are scaled into physical coordinates, $\mathbf{c}_k = (z_k \Delta z, y_k \Delta y, x_k \Delta x)$, so the metric is natively in micrometers. With $\gamma =$ `neighbor_island_fraction` (default **0.1**) the neighbor count is

$$\kappa = \min(\max(\lceil \gamma n_i \rceil - 1, 1), n_i - 1) \quad (1.11)$$

and, writing $\mathcal{N}_\kappa(k)$ for the κ nearest centroids to $\mathbf{c}_k$ *excluding* $\mathbf{c}_k$ itself,

$$D_I = \left\langle \frac{1}{n_i \kappa} \sum_{k=1}^{n_i} \sum_{l \in \mathcal{N}_\kappa(k)} \|\mathbf{c}_k - \mathbf{c}_l\| \right\rangle_i \quad (1.12)$$

NaN whenever a frame holds fewer than two components — the ordinary case for a single segmented nucleus.

NOT COMPARABLE WITH THE 2D METRIC

The 2D implementation averages the $\kappa + 1$ smallest entries of each row of the distance matrix, which include the diagonal zero, where equation (1.12) averages the κ excluding it — so the two average over different sets and stand exactly $\kappa/(\kappa + 1)$ apart before anything else. The 2D version also works in isotropic pixel units and rescales at the end; the anisotropy of a Z-stack leaves no single ratio to apply afterwards, so this scales each axis by its own spacing beforehand.

1.12 Structural correlation length ξ_I

COLUMN Structural Correlation Length · **UNIT** $\mu\mathbf{m}$

Computed on the *raw* voxels — never the binary and never the mask, even when a segmentation replaced thresholding. Normalizing to zero mean and unit variance,

$$B'(x, y, z, i) = \frac{B(x, y, z, i) - \bar{B}(i)}{\sigma_B(i)} \quad (1.13)$$

the autocorrelation follows from the Wiener-Khinchin theorem, using the full 3D transform with no zero padding, so it is the *circular* autocorrelation:

$$G(\Delta, i) = \frac{1}{N_v} \mathcal{F}^{-1} \left[|\mathcal{F}\{B'\}|^2 \right](\Delta), \quad G(\mathbf{0}, i) \simeq 1 \quad (1.14)$$

The 3D field is then radially averaged over shells of physical radius. With $\delta = \min(\Delta x, \Delta y, \Delta z)$ and shell edges $r_j = j\delta$ bounded by

$$r_{\max} = \min\left(\frac{n_z}{2} \Delta z, \frac{n_y}{2} \Delta y, \frac{n_x}{2} \Delta x\right) \quad (1.15)$$

each shell $\mathcal{S}_j = \{\Delta : r_j \leq \|\Delta\| < r_{j+1}\}$ reports both its mean correlation and *its own mean radius*:

$$g_j = \frac{1}{|\mathcal{S}_j|} \sum_{\Delta \in \mathcal{S}_j} G(\Delta), \quad \rho_j = \frac{1}{|\mathcal{S}_j|} \sum_{\Delta \in \mathcal{S}_j} \|\Delta\| \quad (1.16)$$

Three departures from the 2D branch, each deliberate: shells are binned on *physical* distance so anisotropic voxels do not distort the profile; $r_{\max}$ is bounded by the smallest physical half-extent across all three axes rather than a *y*-index assumption; and each shell is anchored at ρ_j rather than at the bin's left edge. A shell's population grows as r^2 , so its mean radius lies well above its left edge, and edge-anchoring makes the profile appear to decay faster than it does. A consequence of ρ_j is that reported lengths are *not* integer multiples of the voxel size, unlike in 2D.

With threshold $\tau = e^{-1}$, let j^* be the smallest j with $g_j > \tau$ and $g_{j+1} \leq \tau$ — the first downward crossing. Then

$$\xi_I(i) = \begin{cases} \rho_{j^*}, & |g_{j^*} - \tau| < |g_{j^*+1} - \tau| \\ \rho_{j^*+1}, & \text{otherwise} \end{cases} \quad \xi_I = \langle \xi_I(i) \rangle \quad (1.17)$$

THIS IS NOT THE REFERENCE-RELEASE DEFINITION

Equation (1.17) snaps to whichever adjacent shell's *correlation value* lies closer to τ , and can therefore return a radius at which the correlation is still *above* threshold. The BARCODE 2.0 release that produced the published reference CSVs took the first radius at which $g \leq \tau$ — always the outer point — so reported lengths here are systematically **smaller**. This matches the current 2D branch verbatim, by design, so that 2D and 3D agree with each other rather than with the reference.

$\xi_I(i) = \text{NaN}$ when the volume is constant, when $r_{\max} \leq \delta$, or when no downward crossing exists within $r_{\max}$ — the last being the "correlation length exceeds the field of view" case that raises flag digit 3.

A subtlety: an empty shell yields $g_j = \text{NaN}$, and NaN comparisons are false, so a gap in the profile silently blocks a crossing at that index. This is a second route to flag 3 beyond the documented one.

2 Intensity Distribution Branch

Metrics 2.1–2.6 restate the published branch with a volume in place of a plane; since the histogram flattens its input, the arithmetic on a given voxel set is identical. Metrics 2.7–2.10 are new: they are the branch's first *extensive* quantities, scaling with the amount of material rather than describing the shape of the histogram.

2.0 The histogram

Per analyzed timepoint the voxel values are binned into N_b bins (`bin_size`, default **300**) over the data's own range — there is no fixed range and no background exclusion. Writing β_j for the bin centers and c_j for the counts, the probabilities are $p_j = c_j / \sum_l c_l$. Bins with $p_j \leq \eta$ are then *discarded* ($\eta = \text{noise_threshold}$, default 5×10^{-4}) and the survivors renormalized:

$$\mathcal{B} = \{j : p_j > \eta\}, \quad \hat{p}_j = \frac{p_j}{\sum_{l \in \mathcal{B}} p_l}, \quad j \in \mathcal{B} \quad (2.1)$$

This is a *bin-population* filter, not an intensity filter: it removes sparsely populated bins wherever in the range they lie.

All moments are probability-weighted sums over the surviving centers:

$$\mu_i = \sum_{j \in \mathcal{B}} \beta_j \hat{p}_j, \quad \sigma_i = \left(\sum_{j \in \mathcal{B}} (\beta_j - \mu_i)^2 \hat{p}_j \right)^{1/2} \quad (2.2)$$

$$\beta_i^{\text{mode}} = \beta_{\arg \max_j \hat{p}_j}, \quad \beta_i^{\text{med}} = \min \left\{ \beta_j : \sum_{l \leq j} \hat{p}_l \geq \frac{1}{2} \right\} \quad (2.3)$$

Optional mask restriction. With `intensity_use_mask` (default off) the histogram is built only from in-mask voxels; for an instance mask this means *any* non-zero label, so all objects pool. This changes the quantity rather than refining it — on one test volume it moved the maximum kurtosis from **9.93** to **−0.11** — so masked and unmasked runs must not be pooled.

2.1 Maximum kurtosis K

COLUMN Maximum Kurtosis · UNIT dimensionless

$$K_i = \frac{\sum_{j \in \mathcal{B}} (\beta_j - \mu_i)^4 \hat{p}_j}{\sigma_i^4} - 3, \quad K = \text{top}_{10}(K_i) \quad (2.4)$$

The **−3** makes a Gaussian read **0**. Division by σ_i^4 is unguarded, so a constant volume yields $\pm\infty$ or **NaN**.

2.2 Maximum median skewness S_1

COLUMN Maximum Median Skewness · UNIT dimensionless

Pearson's second coefficient. Positive means the bulk lies below the mean — a dim body with a bright tail.

$$S_1(i) = \frac{3(\mu_i - \beta_i^{\text{med}})}{\sigma_i}, \quad S_1 = \text{top}_{10}(S_1(i)) \quad (2.5)$$

2.3 Maximum mode skewness S_2

COLUMN Maximum Mode Skewness · **UNIT** dimensionless

Pearson's first coefficient, referenced to the histogram peak rather than the median, so it responds to the background peak where S_1 responds to the body of the distribution.

$$S_2(i) = \frac{\mu_i - \beta_i^{\text{mode}}}{\sigma_i}, \quad S_2 = \text{top}_{10}(S_2(i)) \quad (2.6)$$

2.4–2.6 Change metrics ΔK , ΔS_1 , ΔS_2

COLUMNS Kurtosis Change, Median Skewness Change, Mode Skewness Change — suffixed (over Z) in xyz mode · **UNIT** dimensionless

The *difference* form of (0.6), in contrast to the binarization branch's ratio:

$$\Delta K = \langle K_i \rangle_{\text{last } n_e} - \langle K_i \rangle_{\text{first } n_e}, \quad \text{and likewise for } S_1, S_2 \quad (2.7)$$

NaN when $N_f < 2$. In `xyz` mode the progression axis is *depth*, not time, and the columns are renamed accordingly — the rename is the safety mechanism, since a depth trend read as a time trend is a silent scientific error.

2.7 Total intensity Φ

COLUMN Total Intensity · **UNIT** a.u. · opt-in

Let $\mathcal{V}_i$ be the set of finite voxel values considered at timepoint i (all of them, or the in-mask subset).

$$\Phi = \left\langle \sum_{u \in \mathcal{V}_i} u \right\rangle_i \quad (2.8)$$

The branch's only genuinely extensive quantity: it doubles when the object doubles, which no shape metric does. Two caveats — it is a raw sum *including background* unless the mask restriction is on, and it is meaningless if the detector clipped, so flag digit 2 should be checked first.

2.8–2.9 Mean intensity $\bar{\Phi}$ and intensity SD σ_Φ

COLUMNS Mean Intensity, Intensity SD · **UNIT** a.u.

$$\bar{\Phi} = \left\langle \frac{1}{|\mathcal{V}_i|} \sum_u u \right\rangle_i, \quad \sigma_\Phi = \left\langle \left[\frac{1}{|\mathcal{V}_i|} \sum_u (u - \bar{u}_i)^2 \right]^{1/2} \right\rangle_i \quad (2.9)$$

The standard deviation is the *population* form (**ddof** = 0). Being per-sample, $\bar{\Phi}$ is invariant to voxel size, unlike ρ_Φ below.

2.10 Intensity density ρ_{Φ}

`COLUMN` Intensity Density (per volume) in a 3D mode, (per area) in a 2D mode · `UNIT` a.u./ μm^3 or a.u./ μm^2

$$\rho_{\Phi} = \left\langle \frac{\sum_{u \in \mathcal{V}_i} u}{|\mathcal{V}_i| v_0} \right\rangle_i \quad (2.10)$$

Signal per unit physical volume on the *analyzed* grid, so unlike $\bar{\Phi}$ it does change with voxel size. **NaN** when the spacing is degenerate. In `xyz` mode the sample size is the in-plane pixel area, the *z* factor being **1**.

3 Optical Flow (3D) Branch

A static z-stack gets **no flow columns at all**. When the branch produces nothing — a single timepoint, or a series shorter than the window — the seven flow columns are dropped from the CSV and the barcode rather than written as all-NaN black stripes. Volumetric modes only; the 2D schema is unchanged, and such a CSV reads back correctly.

PROVISIONAL — TREAT AS WORK IN PROGRESS

The 3D flow branch is functional and wired in, but its **absolute values are not yet validated for publication**. The solver systematically under-reports speed (§3.3) and assumes small displacements (§3.1). Use it for *relative* comparisons within a consistent dataset, not for an absolute $\mu\text{m/s}$. Three of the seven columns are also not comparable with their 2D namesakes, and Speed only relatively.

Seven columns matching the published 3.1–3.7, computed by a different solver on a different temporal unit.

3.1 The solver

Not Farnebäck — no three-dimensional equivalent exists in the library — but Lucas–Kanade with Gaussian-derivative gradients and a Gaussian-weighted structure tensor. It is a **differential** method and assumes **small displacements** between consecutive volumes; there is no coarse-to-fine pyramid, so large motion is under-read. It fits BARCODE's philosophy — CPU-only, no GPU, no training, runnable by any user — the same trade the 2D Farnebäck branch makes. Each analyzed timepoint requires a *contiguous* window of

$$W = 6\sigma_t + 1 \quad (3.1)$$

volumes centered on it, $W = 7$ at the default $\sigma_t = 1$. The temporal derivative is taken across that window; **only the central volume is then retained**, and every spatial derivative and the whole structure tensor are formed from it alone. Writing $\mathbf{v}$ for the unknown velocity, ∇I for the spatial gradient and I_t for the temporal derivative, the weighted normal equations are

$$\underbrace{\left(\sum_{\mathcal{W}} w \nabla I \nabla I^\top\right)}_{\mathbf{M}} \mathbf{v} = - \sum_{\mathcal{W}} w \nabla I I_t \quad (3.2)$$

where w is an isotropic Gaussian of width σ_w (default 4 voxels) and $\mathbf{M}$ is the 3×3 structure tensor. The system is solved in closed form with machine epsilon added to $\det \mathbf{M}$ as a divide-by-zero guard. The *reliability* at each voxel is the smallest eigenvalue,

$$r = \lambda_{\min}(\mathbf{M}) \quad (3.3)$$

Velocities emerge in voxels per frame and are converted to $\mu\text{m s}^{-1}$:

$$v_x = \frac{u_x \Delta x}{\Delta t}, \quad v_y = \frac{u_y \Delta y}{\Delta t}, \quad v_z = \frac{u_z \Delta z}{\Delta t} \quad (3.4)$$

Where the y -up convention lives. BARCODE reports directions y -up while the array indexes y downward. The 2D branch reconciles this by reversing the array axis *and* negating the component, leaving the field self-consistent. The 3D branch instead leaves the field untouched and negates the y component of the **mean unit vector only**, where the azimuth is reported (§3.5) — the single place a sign convention is observable.

Confining it there is what makes the spatial operators correct. Negating a component without reversing its axis yields a field belonging to no frame at all: for any subsequent derivative $\partial v_y / \partial y = -\partial v_y / \partial (j\Delta y)$, which is why confining the convention to the reported direction keeps §3.8 and §3.9 correct.

3.2 The validity mask — and where it deliberately does not apply

A requested center c survives only if $\lfloor W/2 \rfloor \leq c \leq N_T - 1 - \lfloor W/2 \rfloor$. Since the selector (0.2) always includes 0 and $N_T - 1$, **the first and last three timepoints of any series never produce flow**, and a series shorter than W skips the branch entirely — returning all-**NaN** results with flag digit 4 *unset*.

Two filters define the per-voxel validity mask $\mathcal{M}$: a reliability percentile P (default **50**, so half the voxels are discarded) and, optionally, the segmentation:

$$\mathcal{M} = \{ \mathbf{x} : r(\mathbf{x}) \geq Q_P(r) \} \cap \{ \mathbf{x} \in \text{mask} \} \quad (3.5)$$

where Q_P is the P^{th} percentile. The mask is applied differently by metric type, which is the crux of the design: **pointwise** metrics use the masked field; **spatial-derivative** metrics are evaluated on the full field and masked only at the reduction, since puncturing the field before differentiating would make the derivative describe the hole pattern; and the **correlation length** ignores both filters entirely.

3.3 Speed v

COLUMN Speed · UNIT $\mu\text{m s}^{-1}$

$$v = \left\langle \frac{1}{|\mathcal{M}|} \sum_{\mathbf{x} \in \mathcal{M}} \sqrt{v_x^2 + v_y^2 + v_z^2} \right\rangle_{\text{windows}} \quad (3.6)$$

Genuinely 3D: out-of-plane motion contributes. A fully masked window yields **NaN** and is excluded from the average rather than counted as zero.

Speeds are a lower bound. The solver is gradient-based, so recovered speed is a fixed fraction of true speed when features are comparable in size to σ_{xyz} : measured gain ≈ 0.55 at a 2-voxel feature scale and ≈ 0.92 at 6 voxels, at the default $\sigma_{xyz} = 3$. The gain is linear in speed, so relative comparisons across a dataset are sound while absolute values are not.

3.4 Speed change Δv

COLUMN Speed Change · UNIT $\mu\text{m s}^{-1}$

The difference form of (0.6) over *windows*. **NaN** for a single window — and because the default stride plus the edge exclusion frequently leaves exactly one surviving center, **NaN** here is common and expected.

3.5 Mean flow direction θ

COLUMN Mean Flow Direction · **UNIT** rad, range $(-\pi, \pi]$

This is the ***xy* azimuth only**. Elevation is deliberately discarded — folded into §3.6 instead — so that the column remains comparable with 2D runs. There is no elevation or polar-angle column.

Two-stage vector averaging, never an angle average. Per window the field is normalized to 3D unit vectors and averaged componentwise over valid voxels; the mean unit vector is *not* renormalized, so its magnitude carries the coherence:

$$\bar{\mathbf{e}} = \frac{1}{|\mathcal{M}|} \sum_{\mathbf{x} \in \mathcal{M}} \frac{\mathbf{v}(\mathbf{x})}{\|\mathbf{v}(\mathbf{x})\|}, \quad \theta = \text{atan2} \left(-\langle \bar{e}_y \rangle, \langle \bar{e}_x \rangle \right) \quad (3.7)$$

The minus sign in (3.7) is the whole of the *y*-up convention (§3.1). It cannot disturb a derivative, and σ_θ below takes the magnitude of $\bar{\mathbf{e}}$, which the sign leaves unchanged.

Versus 2D. Two differences, both following from the extra dimension and the validity mask. The unit vectors here are 3D, so a voxel moving largely out of plane contributes a shorter in-plane projection and its azimuth is damped, whereas 2D unit vectors always have in-plane length exactly **1**. And 2D averages over *every* pixel, including motionless ones, where $\text{atan2}(0, 0) = 0$ contributes $(1, 0)$ and pulls the result toward $+\mathbf{x}$; this averages over valid voxels only, having a reliability mask with which to define them. The same statistic over different populations — read them separately.

3.6 Directional spread σ_θ

COLUMN Directional Spread · **UNIT** rad

Spherical dispersion from the resultant length of the *full* 3D unit vectors, so out-of-plane scatter widens it where §3.5 is purely azimuthal:

$$R = \|\bar{\mathbf{e}}\|, \quad \sigma_\theta = \left\langle \sqrt{-2 \ln \tilde{R}} \right\rangle, \quad \tilde{R} = \min(\max(R, 10^{-12}), 1) \quad (3.8)$$

$R \rightarrow 1$ is coherent flow ($\sigma_\theta \rightarrow 0$); $R \rightarrow 0$ is isotropic scatter. The clip bounds σ_θ at ≈ 7.434 rad and makes a perfectly coherent window read exactly zero.

The resultant uses a NaN-*propagating* sum by design: a NaN-tolerant sum would give $R = 0$ for a fully masked window, which reads as perfectly isotropic flow and yields a large but finite spread for a window containing no data at all. The 2D branch is unclipped, so $R = 0$ gives $+\infty$ and an R marginally above **1** gives **NaN**.

3.7 Velocity correlation length ξ_v

COLUMN Velocity Correlation Length · **UNIT** $\mu\mathbf{m}$

Evaluated on the *full, unmasked, unfiltered* field. Summing the Wiener–Khinchin transform over the three components and normalizing by the mean square speed,

$$C(\Delta) = \frac{\langle \mathbf{v}(\mathbf{x}) \cdot \mathbf{v}(\mathbf{x} + \Delta) \rangle}{\langle \|\mathbf{v}\|^2 \rangle} = \frac{1}{N_v \langle \|\mathbf{v}\|^2 \rangle} \sum_{a \in \{x,y,z\}} \mathcal{F}^{-1} [|\mathcal{F}\{v_a\}|^2] \quad (3.9)$$

with $C(\mathbf{0}) = 1$. The radial averaging of (1.16) and the threshold-crossing rule of (1.17) are then applied verbatim, with $\tau = 0.5$ in place of e^{-1} , so the same divergence from the reference release applies. $\xi_v = \langle \xi_v(i) \rangle$ over windows.

A window is **NaN** when the field is identically zero, when $r_{\max} \leq \delta$, or when C never crosses **0.5** within $r_{\max}$. Any of the three raises flag digit 4, not only the last.

Because (3.9) depends on each component only through $|\mathcal{F}\{v_a\}|^2$, it is invariant under $v_a \rightarrow -v_a$ — which is why ξ_v escapes the defect of (3.4).

3.8 Divergence $\nabla \cdot \mathbf{v}$

COLUMN Divergence · **UNIT** $\mathbf{s}^{-1}$ in a volumetric mode; blank in 2D, where the quantity differs (below)

$$\nabla \cdot \mathbf{v} = \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y} + \frac{\partial v_z}{\partial z} \quad (3.10)$$

by second-order central differences on the physical grid (one-sided at the borders), evaluated on the full field, masked at the reduction, and averaged over windows. Positive indicates local expansion.

Verified against analytic fields: $\nabla \cdot (\mathbf{k}\mathbf{r}) = 3\mathbf{k}$ exactly, zero for uniform flow and zero for rigid rotation, and invariant under proper rotations. The field reaches this operator in the solver's own consistent frame (§3.1).

Versus 2D. Two different quantities sharing a column name; no rescaling converts one into the other. The 2D metric is the spatial mean divergence of the *cumulative sum of unit-vector fields*, taken at the last frame pair only — a coherent measurement of accumulated *direction* over a series, in $\mu\mathbf{m}^{-1}$, dependent on how many pairs preceded it. Equation (3.10) is a window-mean $\nabla \cdot \mathbf{v}$ of the physical velocity field, in $\mathbf{s}^{-1}$. Do not pool them.

3.9 Curl Ω

COLUMN Curl · **UNIT** $\mathbf{s}^{-1}$ in a volumetric mode; blank in 2D, where the quantity differs (below)

In a plane the curl is a signed scalar, which is what the 2D branch reports. In three dimensions it is a *vector* and there is one column, so this branch reports the mean of its *magnitude*:

$$\nabla \times \mathbf{v} = \begin{pmatrix} \partial_y v_z - \partial_z v_y, & \partial_z v_x - \partial_x v_z, & \partial_x v_y - \partial_y v_x \end{pmatrix}, \quad \Omega = \left\langle \|\nabla \times \mathbf{v}\| \right\rangle \quad (3.11)$$

Being a magnitude, $\Omega \geq 0$ and carries **no handedness**: clockwise and anticlockwise vortices give the same value, and counter-rotating regions add in the mean rather than canceling, as they do in the 2D signed scalar. Neither convention converts into the other.

Verified against analytic fields: $\|\nabla \times \mathbf{v}\| = 2\omega$ everywhere for solid-body rotation at rate ω , zero for uniform flow, and invariant under proper rotations. (A plain transpose is a *reflection*, which flips the curl sign, so it is not a valid check.)

4 Volumetric Branch

Sections 1–3 are the **three branches** of published BARCODE, restated for a volume. This is the fourth branch: everything the volume makes possible that a flat image cannot give you. It is one branch, not seven — the families inside it are grouped by **what they require**, because that is the only thing to decide before a run. Almost all are **opt-in**:

Family	§	Default	Needs a segmentation?	Also needs
Surface mesh	4.1–4.11	off	no — falls back to the thresholded volume	isotropic grid
Curvature	4.12–4.18	on within meshing	no (rides on the mesh)	the mesh
Object size distribution	4.19–4.22	off	no	—
Depth profile	4.23–4.28	off	no	—
In-mask intensity	4.29–4.37	off	yes	—
Packing topology	4.38–4.44	off	instance mask	isotropic grid
Object rows	4.45	by <code>row_axis</code>	instance mask	—
Range provenance	4.46	off	no	—

Sixteen of these columns need cubic voxels and are absent without them — the whole mesh and curvature set, and all of packing. §P.4 lists which metrics depend on the grid and what to do about each.

A single volume is a normal input. Every *Change* metric is then NaN — a change needs two points on the progression axis — and everything else is live.

Runs without a segmentation

4.1 How the surface is built

Eight columns describing a triangulated surface. Requires an isotropic grid; opt-in, though the `xyzt` schema advertises the columns. A **missing segmentation no longer prevents meshing** — the branch falls back to the *binarized volume* and says so, which is a threshold surface rather than the object. Surface extraction is a CGAL Delaunay-refinement mesher at `mesh_isovalue` (default **0.5**) on the binary volume, with radius bound `mesh_maxrad`; there is deliberately no marching-cubes fallback, which would produce meshes not comparable with the reference MATLAB pipeline. The binary is reduced to its largest 26-connected component and padded by one voxel of background so a boundary-touching object still closes; the mesh — not the mask — is then smoothed by Laplacian-HC.

The isovalue is the boundary a binary mask means. `mesh_isovalue` defaults to **0.5** — the boundary between the last foreground voxel and the first background one, and the most accurate value on rounded objects (within ~0.1% of the exact sphere volume down to a 16-voxel radius). A higher value pulls the surface inward onto foreground-voxel centers and shrinks small objects; setting **0.99** with `mesh_matlab_compat` reproduces the MATLAB pipeline; the measured agreement figures for that setting are in `docs/volumetric_manual.md` §4.1.

Let the mesh be $(\mathcal{V}, \mathcal{T})$ with vertices $\mathbf{p} \in \mathbb{R}^3$ in micrometers and triangles $t = (\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2)$.

4.2 Mesh volume V_m

COLUMN Mesh Volume · UNIT $\mu\mathbf{m}^3$

By the divergence theorem, as a sum of signed tetrahedra on the origin:

$$V_m = \left| \frac{1}{6} \sum_{t \in \mathcal{T}} \mathbf{p}_0 \cdot (\mathbf{p}_1 \times \mathbf{p}_2) \right| \quad (4.1)$$

The sign encodes surface orientation and is retained as a diagnostic. V_m is deliberately distinct from I_{tot}^Q of §1.10: meshing smooths the surface, so the two differ by a few percent, and §4.8 reports exactly that discrepancy.

4.3 Mesh surface area A_m

COLUMN Mesh Surface Area · UNIT $\mu\mathbf{m}^2$

$$A_m = \sum_{t \in \mathcal{T}} a_t, \quad a_t = \frac{1}{2} \|(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)\| \quad (4.2)$$

4.4 Sphericity Ψ

COLUMN Sphericity · UNIT dimensionless

Wadell sphericity — the area of a sphere of equal volume over the actual area. Exactly **1** for a sphere, below **1** otherwise; **NaN** when $A_m \leq 0$.

$$\Psi = \frac{\pi^{1/3} (6V_m)^{2/3}}{A_m} \quad (4.3)$$

4.5 Equivalent sphere radius R_{eq}

COLUMN Equivalent Sphere Radius · UNIT $\mu\mathbf{m}$

$$R_{eq} = \left(\frac{3V_m}{4\pi} \right)^{1/3} \quad (4.4)$$

4.6 Mesh height h

COLUMN Mesh Height · UNIT $\mu\mathbf{m}$

The z extent of the face *centroids* $\mathbf{q}_t = \frac{1}{3}(\mathbf{p}_0 + \mathbf{p}_1 + \mathbf{p}_2)$, not of the vertices:

$$h = \max_t q_{t,z} - \min_t q_{t,z} \quad (4.5)$$

Deliberate parity with the reference MATLAB implementation. Centroids lie inside the surface, so h is systematically shorter than the true z extent — which is the sole reason a perfect sphere reports a lateral/axial ratio near **1.08** rather than **1**.

4.7 Lateral/Axial ratio α

COLUMN Lateral/Axial Ratio · **UNIT** dimensionless

Projecting the largest connected component along z and fitting the ellipse with the same normalized second central moments, with major and minor axes $\ell_{\text{maj}}, \ell_{\text{min}}$:

$$\alpha = \frac{\ell_{\text{maj}} + \ell_{\text{min}}}{2h} \quad (4.6)$$

Lateral size over axial size: $\alpha > 1$ is oblate, $\alpha \approx 1$ round. Dimensionless and hence size-independent. **NaN** if $h = 0$ or the mask is absent.

4.8 Mesh volume ratio Υ

COLUMN Mesh Volume Ratio · **UNIT** dimensionless

The mesh volume against the raw voxel-counted volume of the same mask, where n_{obj} is the object's voxel count:

$$\Upsilon = \frac{V_m}{n_{\text{obj}} v_0} \quad (4.7)$$

A fidelity check, not a shape descriptor. $\Upsilon \approx 1$ means the mesh faithfully represents the mask. The default radius bound is tuned for objects ~ 200 voxels across and loses roughly half the volume of a thin object: measured $\Upsilon = 0.51, 0.73, 0.78, 0.80, 0.81$ at `mesh_maxrad = 5.0, 3.0, 2.0, 1.5, 1.0`. Scale the bound to the object's *thinnest* dimension, roughly `depth/8`, and check Υ on any dataset that is not a nucleus.

4.9 Solidity Σ

COLUMN Solidity · **UNIT** dimensionless

Both sides are *voxel counts* — deliberately, so that meshing shrinkage does not contaminate what is meant to be a pure convexity measure. With $\mathcal{H}$ the convex hull taken over voxel centers,

$$\Sigma = \frac{n_{\text{obj}}}{n_{\mathcal{H}}} \quad (4.8)$$

$\Sigma = 1$ is convex, lower is more lobed. Degenerate inputs (fewer than four object voxels, or a hull failure on collinear or planar input) report the object as fully solid.

A voxel is a box, so "the hull of the object" has more than one reasonable reading. Against 187 stored MATLAB values the voxel-center convention used here sits $\approx 1.5\%$ above the reference MATLAB column and the voxel-corner convention $\approx 1.2\%$ below it, with MATLAB between — three readings of one shape rather than one right answer. Centers were chosen

for exact agreement with a sibling analysis package, so Σ is directly comparable there and offset by $\approx 1.5\%$ against MATLAB. This is a source-comment claim, not pinned by a test in this repo.

4.10 Concavity — removed

There is **no Concavity** column. It was $1 - \Sigma$ and carried no information Solidity does not, so it was dropped from every schema — the field mesh family and the per-object rows alike. Read lobedness directly off **Solidity** (§4.9): lower is more lobed. For a genuine surface-concavity measure use **Concave Area Fraction** (§4.16), a different quantity entirely — the area fraction of concave faces, not a hull-fill ratio.

4.11 How multiple objects and timepoints are combined

One object is meshed per analyzed timepoint — the largest connected component — and every mesh field is reduced by the finite-only mean (0.3) over timepoints. Note this averages the *extensive* quantities V_m and A_m as well as the intensive ones. Per-object meshing of a labeled field is implemented (`mesh_field.py`) but has no pipeline call site and provides no scalar reduction; per-object shape comes from §4.45 instead.

4.12 How curvature is estimated

Principal curvatures over the mesh surface, estimated by finite differences of vertex normals along triangle edges, solved per face in least squares, projected into per-vertex tangent frames and accumulated with mixed-Voronoi area weights — neither a quadric fit nor a cotangent Laplacian. Verified bit-for-bit against the reference MATLAB toolbox.

Winding is corrected first: if (4.1) is signed negative the faces are flipped, since an inward-wound mesh would invert every sign. Per-vertex principal curvatures are averaged to their three corners, giving per-face $\kappa_{\min,t} \leq \kappa_{\max,t}$ and

$$H_t = \frac{1}{2}(\kappa_{\min,t} + \kappa_{\max,t}), \quad \mathcal{K}_t = \kappa_{\min,t} \kappa_{\max,t} \quad (4.9)$$

the pointwise mean and Gaussian curvatures. Units are μm^{-1} ; with outward normals a sphere of radius R gives $H = +1/R$, so **concave is negative**. $\mathcal{K}_t$ is computed but never emitted.

4.13 Which faces are excluded — by default, none

By default no faces are excluded — curvature is measured over the whole surface, and top and bottom caps are not clipped. Two exclusion rules exist, both **off by default**, and both must be switched on deliberately with a supplied threshold — heavily gated because discarding surface changes every curvature number:

Setting	Default	Excludes
<code>curvature_exclude_caps</code>	False	Faces whose centroid falls in the lowest or highest 0.1 μm z bin.
<code>curvature_outlier_limit</code>	0.0 (off)	Faces with $ H $ above the limit, in μm^{-1} . MATLAB uses 2.0 , a radius of curvature below 0.5 μm .

The two answer different questions. **Cap exclusion is anatomical**: where a segmentation is clipped by the ends of the imaged stack the resulting flat cap is an artifact of the acquisition, not a surface of the object. Turn it on when flag digit 6 is raised. For an object sitting entirely inside the volume it discards real surface, which is why it is off by default. **The outlier limit is numerical**: a face reporting a radius of curvature well below the

mesh's own edge length is a badly conditioned triangle rather than fine structure. With the default of **0** every face counts, so a mesh with degenerate triangles can pull $\langle \mathbf{H} \rangle$ around — check Mesh Volume Ratio (§4.8) if a value looks implausible.

When a limit is set the outlier decision is made **once, on $\mathbf{H}$** , and reused for the minimum and maximum, so all three means always run over an identical face set.

Reproducing MATLAB. Setting `curvature_exclude_caps = True` and `curvature_outlier_limit = 2.0` reproduces the MATLAB behavior and the parity recorded in §4.12. The defaults measure the whole surface — the more defensible starting point for an object fully inside the stack — so discarding surface is opt-in.

With both exclusion rules off, the two ratios (§4.15, §4.16) and the three means share a single denominator — the whole surface, less any face whose principal curvatures were non-finite.

4.14 Mean curvature $\langle \mathbf{H} \rangle$

COLUMN Mean Curvature $\langle \mathbf{H} \rangle$ · **UNIT** $\mu\mathbf{m}^{-1}$

$$\langle \mathbf{H} \rangle = \frac{\sum_{t \in \mathcal{T}^*} a_t \mathbf{H}_t}{\sum_{t \in \mathcal{T}^*} a_t} \quad (4.10)$$

Two averages are stacked: $\mathbf{H}$ is already a *local* mean of the two principal curvatures, and (4.10) then averages it over the usable surface — hence the $\langle \cdot \rangle$ notation rather than the bare name. **NaN** when the usable area is zero.

4.15–4.16 Invagination ratio and concave area fraction

COLUMNS Invagination Ratio, Concave Area Fraction · **UNIT** dimensionless, **0–1**

Faces are classified in this precedence — the non-finite test wins outright, and after it $\kappa_{\max} \leq 0$ wins, so a face with both curvatures zero is concave:

$$\text{class}(t) = \begin{cases} \text{unclassified,} & \kappa_{\min,t} \text{ or } \kappa_{\max,t} \text{ non-finite} \\ \text{concave,} & \kappa_{\max,t} \leq 0 \\ \text{saddle,} & \kappa_{\max,t} > 0 \text{ and } \kappa_{\min,t} \leq 0 \\ \text{convex,} & \text{otherwise} \end{cases} \quad (4.11)$$

and with the usable set $\mathcal{T}^*$ of (4.10) — every face that is neither excluded by §4.13 nor unclassified — as the common denominator,

$$\text{Invagination} = \frac{\sum_{t \in \mathcal{T}^*, \text{concave or saddle}} a_t}{\sum_{t \in \mathcal{T}^*} a_t}, \quad \text{Concave} = \frac{\sum_{t \in \mathcal{T}^*, \text{concave}} a_t}{\sum_{t \in \mathcal{T}^*} a_t} \quad (4.12)$$

The concave fraction is a strict subset of the invagination ratio, so it is always the smaller of the two.

The *unclassified* class is a deliberate departure from the reference MATLAB implementation. A non-finite principal curvature would otherwise fall through to *convex*, counting a face that could not be measured as evidence against invagination; removing it from the denominator as well as the numerator makes both ratios describe the surface that was actually classifiable.

4.17–4.18 Minimum and maximum curvature

COLUMNS Minimum Curvature, Maximum Curvature · **UNIT** μm^{-1} · opt-in

$$\kappa_{\min} = \frac{\sum_{t \in \mathcal{T}^*} a_t \kappa_{\min,t}}{\sum_{t \in \mathcal{T}^*} a_t}, \quad \kappa_{\max} = \frac{\sum_{t \in \mathcal{T}^*} a_t \kappa_{\max,t}}{\sum_{t \in \mathcal{T}^*} a_t} \quad (4.13)$$

These are area-weighted *means* of the per-face principal curvatures, not global extrema. They exist because (4.10) averages $\kappa_{\min}$ and $\kappa_{\max}$ together, so a saddle — sharply curved in both directions — averages towards zero and reads as flat. A surface can be highly structured and still report $\langle \mathbf{H} \rangle \approx \mathbf{0}$; (4.13) says how structured.

4.19–4.22 Object size distribution

COLUMNS Island Count, Island Volume SD, Island Volume Skewness, Median Island Volume · **UNIT** dimensionless · opt-in, volumetric only

The binarization branch gives the largest, mean and total object size but cannot say whether objects are uniform or wildly unequal — one dominant object plus debris has the same mean as several even ones. Writing $s_k = |\mathcal{R}_k(i)|$ and $\bar{s}_i$ for their mean,

$$n = \langle n_i \rangle, \quad \text{SD} = \frac{1}{N_v} \left\langle \left[\frac{1}{n_i} \sum_k (s_k - \bar{s}_i)^2 \right]^{1/2} \right\rangle, \quad \text{Med} = \frac{1}{N_v} \langle \text{median}_k s_k \rangle \quad (4.14)$$

$$\text{Skew} = \left\langle \frac{1}{n_i} \sum_k \left(\frac{s_k - \bar{s}_i}{\sigma_{s,i}} \right)^3 \right\rangle \quad (4.15)$$

Standard deviations are the population form. The skewness is **NaN** unless a frame holds more than two objects with non-zero variance; the SD is **0**, not **NaN**, for a single object. All four are means of per-timepoint statistics, *not* statistics pooled over timepoints.

4.23–4.28 Depth profile

COLUMNS Maximal Area Slice Index, Maximal Area Slice Depth, Maximal Area Slice Area · **UNITS** slice, μm , fraction of FOV · opt-in

The only metrics that say *where* in depth something is, rather than reducing the stack to one number. With a_z the foreground fraction of slice z ,

$$a_z = \frac{1}{n_x n_y} \sum_{x,y} b(x, y, z, i), \quad z^* = \arg \max_z a_z, \quad d^* = z^* \Delta z \quad (4.16)$$

reported as $\langle z^* \rangle$, $\langle d^* \rangle$ and $\langle a_{z^*} \rangle$. The first maximum wins on a tie, and the time-average makes the reported index fractional. The index is into the *analyzed* stack, and d^* is measured from the first analyzed slice — a shape descriptor, not an absolute stage position.

This family also raises **flag digit 6**, the union over analyzed timepoints of whether foreground touches any of the six faces of the analyzed volume. When it fires, every size, shape and curvature metric describes a truncated object, which is not recoverable from the numbers themselves.

Requires a segmentation

4.29–4.37 In-mask intensity

COLUMNS In-Mask MFI, SD, CV, Skewness, Entropy, Normalized Entropy, Fraction Above 2x Median ·
opt-in, requires a segmentation

How signal is distributed *inside* each object — the clustering readout. The intensity branch describes whatever voxels it is given, usually including background; a uniformly filled nucleus and one with bright foci share a mean and differ sharply in CV and entropy.

For object $\mathbf{o}$ with raw voxel values $\{u\}$, mean μ_o and population SD σ_o :

$$\text{CV}_o = \frac{\sigma_o}{\mu_o}, \quad \gamma_o = \frac{1}{|o|} \sum_u \left(\frac{u - \mu_o}{\sigma_o} \right)^3, \quad \Phi_o^{2\times} = \frac{|\{u : u > 2 \text{ median}(u)\}|}{|o|} \quad (4.17)$$

Entropy alone uses a per-object rescaling $\tilde{u} = (u - \min u) / (\max u - \min u)$, binned into N_e bins (default **64**) over the *fixed* range $[0, 1]$ so that objects share bin widths:

$$S_o = - \sum_{j: q_j > 0} q_j \log_2 q_j, \quad \hat{S}_o = \frac{S_o}{\log_2 N_e} \quad (4.18)$$

Base 2, so the unit is bits with maximum $\log_2 64 = 6$. $\hat{S}_o = 1$ is a uniformly filled object, falling toward **0** as signal concentrates. Runs are comparable only at equal N_e , since it sets both the achievable entropy and the denominator.

Only entropy is rescaled — deliberately. The source MATLAB rescales before every statistic. CV and skewness are already scale-invariant and the $-\min$ shift of an affine map distorts them; and the bright fraction degenerates entirely, since a punctate object's rescaled median is **0**, exactly where the metric matters most.

Objects below `mask_intensity_min_voxels` (default **8**) are skipped, as is any object of constant value. Every statistic is the **unweighted** mean over objects, then over timepoints, so one large object does not outvote the rest. CV and the bright fraction are **NaN** when the object's mean or median is not positive; skewness needs at least three voxels and non-zero variance.

4.38–4.44 Packing topology

COLUMNS Mean Contact Number, Contact Number SD, Hexagonal Fraction · **UNIT** dimensionless · opt-in,
requires an instance segmentation

REQUIRES AN ISOTROPIC GRID; LEFT EMPTY WITHOUT ONE

Contact is counted in **voxel faces** and gaps bridged by a **voxel dilation**. On a non-cubic grid a **z**-normal face spans a different physical area from an **xy**-normal one, so the same physical interface passes or fails the threshold on its orientation alone. Mean island anisotropy (§1.8) meets the same difficulty and escapes it by measuring in physical coordinates; here no rescaling repairs it, because a single voxel distance cannot stand for one physical distance on a non-cubic grid. Above a 1% anisotropy ratio the family is left empty and its columns never reach the CSV or barcode. The general rule: **a metric defined only on a cubic grid must not reach the output**.

Nothing else in BARCODE describes the *topology* of a packing. In a space-filling monolayer sizes and separations are near-uniform and it is the neighbor-number distribution that changes. The family refuses rather than falling back: with a boolean mask, connectivity labeling would fuse a confluent field into one component and report a contact number of zero, which reads as a measurement rather than a misconfiguration.

Labels are first dilated by **d** voxels to bridge thin background gaps. Two labels are adjacent if they meet across a shared *face* — 6-connectivity, so edge- and corner-touching objects are deliberately not neighbors — and the pair survives only if the shared face count reaches **m** (default **5**, floored at **1**, so **m = 0** does not disable the filter):

The face-only guarantee holds only at $d = 0$. At the default `packing_contact_dilation_vox = 1`, two labels meeting along an *edge* each grow into the shared diagonal and afterwards share a genuine 6-connected face running the whole length of that edge — of order 30 voxels for a 30-voxel-tall cell, far above **m**, which cannot filter it because the counts in (4.19) are measured on the *dilated* volume. So with bridging on, edge-touching objects **do** become neighbors, and the face counts are not raw contact areas. Set **d = 0** for strict face adjacency.

$$E = \left\{ \{u, w\} : u \neq w, \left| \{ \mathbf{x} \text{ face-adjacent} : \{L(\mathbf{x}), L(\mathbf{x}')\} = \{u, w\} \} \right| \geq m \right\} \quad (4.19)$$

The contact number of a label is its degree in (**labels**, **E**) — a count of distinct neighbors, not a contact area. Objects reaching the edge are excluded from the *reported set* **I**, though their neighbors still count toward interior degrees. Which faces count is set by `packing_border_mode`, default "xy" — the four lateral faces only, because in a thin slab every object touches the top and bottom and the six-face rule ("all") would exclude the entire field. Border labels are read from the *undilated* labels, so bridging cannot push an interior object onto the border. Then, over the reported set,

$$\bar{C} = \left\langle \frac{1}{|\mathcal{I}|} \sum_{u \in \mathcal{I}} \deg u \right\rangle, \quad \Phi_6 = \left\langle \frac{1}{|\mathcal{I}|} \left| \{u \in \mathcal{I} : \deg u = 6\} \right| \right\rangle \quad (4.20)$$

with the SD the population form over **I**. All three are **NaN**, with a stated reason, when fewer than two objects exist or when every object touches the border. Φ_6 is the more diagnostic of the two: in any space-filling 2D tiling the mean degree is pinned near **6** by topology, while the fraction is not.

Per object

4.45 Object rows

When `row_axis` resolves to `object`, a second and much smaller schema is written to `<name> Objects.csv` — one row per segmented object: 3 base columns (volume, a principal-axis anisotropy needing no mesh, contact number), 5 mesh-shape columns always in the schema but NaN unless `object_mesh` is on, and the 7 in-mask statistics only when `--mask-intensity` is on. So the default is 8 metric columns, 15 with in-mask. How the row axis is chosen is covered under *Row axes* in `docs/volumetric_manual.md`.

Recording what was analyzed

4.46 Range provenance

`COLUMNS Z Range Start/End , T Range Start/End · UNIT slice · opt-in`

Flag digit 5 records *that* the analysis covered part of the data; these record which part, and make per-file ranges representable where a global setting cannot. Both ends are inclusive, matching the settings a user typed. An unrestricted axis reports its full extent rather than **NaN** — "**0** to **53**" and "no range set" are the same statement. Indices are into the *acquired* data, before any isotropic resampling; the two time columns carry the unit label "slice", which is a mild misnomer.

Column index — look up what you have

Every column BARCODE can emit, in every mode and both unit systems, with the section that defines it. Use this when holding a CSV rather than reading front to back.

Two things this makes visible that the per-metric entries do not. **The same measurement changes name between modes** — Maximum Island Area in xyt/xyz is Maximum Island Volume in xyzt, and the five Change metrics gain an (over Z) suffix in xyz because their progression axis is depth, not time. And **every size metric appears twice**: a bare name holding a fraction of the analyzed field, and a ... Quantity name holding a physical size in μm^2 or μm^3 . A given CSV carries one set or the other, never both.

The table lists **metrics** only. Every CSV also opens with three identity columns — File, Channel and Flags — described in §5 (Warnings and flags).

Column in the CSV	Modes	Unit	Section
Connectivity	xyt, xyz, xyzt	fraction of frames	§1.1
Maximum Island Area	xyt, xyz	fraction of FOV	§1.2
Maximum Island Area Quantity	xyt, xyz	μm^2	§1.2
Maximum Island Volume	xyzt	fraction of FOV	§1.2
Maximum Island Volume Quantity	xyzt	μm^3	§1.2
Maximum Void Area	xyt, xyz	fraction of FOV	§1.3
Maximum Void Area Quantity	xyt, xyz	μm^2	§1.3
Maximum Void Volume	xyzt	fraction of FOV	§1.3
Maximum Void Volume Quantity	xyzt	μm^3	§1.3
Maximum Island Area Change	xyt	ratio to initial	§1.4
Maximum Island Area Change (over Z)	xyz	ratio to initial	§1.4
Maximum Island Volume Change	xyzt	ratio to initial	§1.4
Maximum Void Area Change	xyt	ratio to initial	§1.5
Maximum Void Area Change (over Z)	xyz	ratio to initial	§1.5
Maximum Void Volume Change	xyzt	ratio to initial	§1.5
Initial Maximum Island Area	xyt, xyz	fraction of FOV	§1.6
Initial Maximum Island Area Quantity	xyt, xyz	μm^2	§1.6
Initial Maximum Island Volume	xyzt	fraction of FOV	§1.6
Initial Maximum Island Volume Quantity	xyzt	μm^3	§1.6
Initial 2nd Maximum Island Area	xyt, xyz	fraction of FOV	§1.6–1.7
Initial 2nd Maximum Island Area Quantity	xyt, xyz	μm^2	§1.6–1.7
Initial 2nd Maximum Island Volume	xyzt	fraction of FOV	§1.6–1.7
Initial 2nd Maximum Island Volume Quantity	xyzt	μm^3	§1.6–1.7
Mean Island Anisotropy	xyt, xyz, xyzt	—	§1.8
Mean Island Area	xyt, xyz	fraction of FOV	§1.9
Mean Island Area Quantity	xyt, xyz	μm^2	§1.9
Mean Island Volume	xyzt	fraction of FOV	§1.9
Mean Island Volume Quantity	xyzt	μm^3	§1.9
Total Island Area	xyt, xyz	fraction of FOV	§1.10
Total Island Area Quantity	xyt, xyz	μm^2	§1.10
Total Island Volume	xyzt	fraction of FOV	§1.10
Total Island Volume Quantity	xyzt	μm^3	§1.10
Mean Island Separation	xyt, xyz, xyzt	μm	§1.11

Structural Correlation Length	xyt, xyz, xyzt	μm	§1.12
Maximum Kurtosis	xyt, xyz, xyzt	—	§2.1
Maximum Median Skewness	xyt, xyz, xyzt	—	§2.2
Maximum Mode Skewness	xyt, xyz, xyzt	—	§2.3
Kurtosis Change	xyt, xyzt	—	§2.4
Kurtosis Change (over Z)	xyz	—	§2.4
Median Skewness Change	xyt, xyzt	—	§2.5
Median Skewness Change (over Z)	xyz	—	§2.5
Mode Skewness Change	xyt, xyzt	—	§2.6
Mode Skewness Change (over Z)	xyz	—	§2.6
Total Intensity	xyt, xyz, xyzt	a.u.	§2.7
Mean Intensity	xyt, xyz, xyzt	a.u.	§2.8
Intensity SD	xyt, xyz, xyzt	a.u.	§2.9
Intensity Density (per area)	xyt, xyz	$\text{a.u.}/\mu\text{m}^2$	§2.10
Intensity Density (per volume)	xyzt	$\text{a.u.}/\mu\text{m}^3$	§2.10
Speed	xyt, xyzt	$\mu\text{m/s}$	§3.3
Speed Change	xyt, xyzt	$\mu\text{m/s}$	§3.4
Mean Flow Direction	xyt, xyzt	rads	§3.5
Directional Spread	xyt, xyzt	rads	§3.6
Velocity Correlation Length	xyt, xyzt	μm	§3.7
Divergence	xyt, xyzt	1/s	§3.8
Curl	xyt, xyzt	1/s	§3.9
Mesh Volume	xyzt	μm^3	§4.2
Mesh Surface Area	xyzt	μm^2	§4.3
Sphericity	xyzt	—	§4.4
Equivalent Sphere Radius	xyzt	μm	§4.5
Mesh Height	xyzt	μm	§4.6
Lateral/Axial Ratio	xyzt	—	§4.7
Mesh Volume Ratio	xyzt	—	§4.8
Solidity	xyzt	—	§4.9
Mean Curvature $\langle H \rangle$	xyzt	$1/\mu\text{m}$	§4.14
Invagination Ratio	xyzt	—	§4.15
Concave Area Fraction	xyzt	—	§4.16
Minimum Curvature	xyzt	$1/\mu\text{m}$	§4.17

Maximum Curvature	xyzt	1/μm	§4.18
Island Count	xyzt	—	§4.19–4.22
Island Volume SD	xyzt	—	§4.19–4.22
Island Volume Skewness	xyzt	—	§4.19–4.22
Median Island Volume	xyzt	—	§4.19–4.22
Mean Contact Number	xyzt	—	§4.38–4.44
Contact Number SD	xyzt	—	§4.38–4.44
Hexagonal Fraction	xyzt	—	§4.38–4.44
Maximal Area Slice Index	xyzt	slice	§4.23–4.28
Maximal Area Slice Depth	xyzt	μm	§4.23–4.28
Maximal Area Slice Area	xyzt	fraction of FOV	§4.23–4.28
In-Mask MFI	xyzt	a.u.	§4.29–4.37
In-Mask Intensity SD	xyzt	a.u.	§4.29–4.37
In-Mask Intensity CV	xyzt	—	§4.29–4.37
In-Mask Intensity Skewness	xyzt	—	§4.29–4.37
In-Mask Intensity Entropy	xyzt	—	§4.29–4.37
In-Mask Normalized Entropy	xyzt	—	§4.29–4.37
In-Mask Fraction Above 2x Median	xyzt	—	§4.29–4.37
T Range End	xyt, xyz, xyzt	slice	§4.46
T Range Start	xyt, xyz, xyzt	slice	§4.46
Z Range End	xyt, xyz, xyzt	slice	§4.46
Z Range Start	xyt, xyz, xyzt	slice	§4.46
Object Volume	object rows	μm ³	§4.45
Anisotropy	object rows	—	§4.45
Contact Number	object rows	—	§4.45

An object row (<name> Objects.csv , §4.45) also **reuses** the seven In-Mask ... headers above, and — with object_mesh on — five mesh/curvature headers (Mesh Surface Area, Sphericity, Solidity, Lateral/Axial Ratio, Mean Curvature <H>) for a per-object quantity, not the field one their rows above point to. Its identity columns are File , FOV , Object .

Reading the outputs

What a row is, is a setting. `row_axis` chooses between one row per file, timepoint, z-slice or **object**, and the barcode normalizes per column *across rows* — so it decides what the picture compares. An **object** axis emits a second schema to `<name> Objects.csv` — 3 base columns (`Object Volume`, a principal-axis **anisotropy** computed without a mesh, contact number), 5 mesh-shape columns always present but NaN unless `object_mesh` is on, and the 7 in-mask statistics only under `--mask-intensity` (8 metric columns by default, 15 with in-mask). See §4.45 below, and *Row axes* in `docs/volumetric_manual.md` for how an axis is chosen.

What a volumetric run writes

A dataset-level `<name> Summary.csv`, a `<name> Summary Barcode.png`, and a `<name> Settings.yaml` recording the configuration, in the folder `processed`. Several harnesses also write a `... (physical).csv` carrying the $\mu\text{m}^2/\mu\text{m}^3$ Quantity columns beside the normalized one.

The `xyzt` path writes no per-file outputs — no reduced data structures, per-frame graphs or per-file figures. `xyz` differs: an *unmasked* run with *Save Graphs* on writes a `<file> BARCODE Output/` from the 2D branches, while a *masked* one takes a different path and writes none. Enabling meshing with OBJ export additionally writes `<stem> BARCODE Meshes/` beside the input.

Keep the `Settings.yaml`. Several metrics are interpretable only against the settings that produced them — normalized entropy depends on the bin count, correlation lengths on the voxel size, Speed on the frame interval — and the CSV does not carry those values. `record_range_columns` embeds at least the analyzed z/t ranges into the CSV itself (§4.46).

Reading the barcode PNG

Rows are whatever the row axis resolved to — files, timepoints, z-slices or objects (above), columns are metrics, color is the value on the **plasma** colormap. **NaN renders black** — worth knowing before reading a black cell as "low". Scaling is per column, but how each column is scaled depends on its unit, and only some are normalized against the data:

Column type	Color range
<code>fraction of FOV</code> , <code>fraction of frames</code>	fixed $[0, 1]$ — comparable between runs
Mean Flow Direction	fixed $[-\pi, \pi]$
Directional Spread	fixed $[0, \pi]$; larger values clipped in the image only
Mean Island Anisotropy	[1, max]
ratio to initial	data range, always including 1
Curvature, Divergence, Curl, Speed Change	data range, always including 0 (signed)
Other physical units	[0, max]
Remaining dimensionless	data range, always including 0

A column with a **fixed** range can be compared between two barcodes; one scaled to the data cannot, since its extremes come from whichever rows are in that figure. Dynamic ranges are taken across every rendered row, so **adding or removing files recolors those columns**. Read the CSV when comparing across figures.

5 Warnings and flags

FLAGS ARE EMITTED SEMICOLON-JOINED

A row with a restricted z range whose structural correlation failed reads `3;5`, not a single digit. `0` means no flag fired. Any parser reading this column as an integer is wrong on multi-flag rows.

Digit	Meaning	Criterion
0	No flag fired	—
1	Dim channel	$\frac{2}{\epsilon} \overline{B} \leq B_{\min}$ — the darkest voxel is already a sizeable fraction of the average. Judged on the first frame in 2D and the first <i>analyzed</i> volume in <code>xyzt</code> . Never fires in <code>xyz</code> .
2	Saturation	The histogram mode occupies the highest surviving bin in <i>every</i> analyzed timepoint. Makes §2.7 meaningless.
3	Structural correlation failure	$\xi_I(i) = \text{NaN}$ for <i>at least one</i> analyzed timepoint.
4	Velocity correlation failure	$\xi_v(i) = \text{NaN}$ for at least one window. Not raised when the branch was skipped outright.
5	Range restricted	A z or t range was applied — <i>you</i> narrowed the analysis.
6	Field-of-view clipping	Foreground reaches a face of the analyzed volume — the <i>data</i> is cut off.
7	Open mesh surface	A mesh came out with a boundary rather than closed, so V_m , Ψ and the curvature <i>signs</i> are unreliable for that row — an unclosed surface has no well-defined inside.

Digits 3 and 4 fire if *any single* analyzed timepoint failed, the test being NaN-propagating, whereas the metric itself is a NaN-dropping mean. A finite value with its flag raised therefore means "most timepoints worked, at least one did not". Digit 3's documented meaning is "the correlation length exceeds the field of view", which is the usual cause but not the only one — an empty radial shell also blocks a crossing (§1.12).

6 Known limitations and caveats

The current version has the following limitations. None changes how a metric is defined; they are things to know when setting up a run or reading a result.

Open issues

	Issue	Effect
L1	<code>perslice</code> histogram <code>IndexError</code> is unguarded	Fires when no bin clears the noise threshold — routine for a sparse in-mask selection, and the masked <code>xyz</code> path routes through it. Swallowed by the caller, so all six intensity columns read NaN while the run reports success.
L2	"Parse All Channels" notice fires once per process	A module-level flag never reset, so in the GUI every run after the first drops channels with no message.
L3	No test covers the pipeline-level flow sign convention	The flow tests call the divergence and curl operators directly on synthetic fields, so a regression in the field's sign handling would not be caught by CI.
L4	Resampler edge-clamping is unrecorded	Repeating the last plane past the mask overhang avoids injecting background, but nothing in the CSV or provenance records which resampling convention produced a file.
L5	<code>packing_border_mode</code> and <code>row_axis</code> have no GUI control	Reachable only from a YAML or the CLI.
L6	z-range/mask mapping assumes node-aligned grids	A restricted range on a finer mask grid can be misregistered by up to about one acquired slice at the deep end.
L7	The constant-volume guard uses <code>np.ptp</code> , which is NaN-blind	A float volume containing NaN falls through to the mean-relative threshold; the all-False comparison happens to give the right binary, but an <code>np.isfinite</code> check would make the intent explicit.
L8	A very flat object (fewer than three z bins) reports an excluded-cap fraction of 0.0	Indistinguishable from a genuinely unclipped object; curvature is then measured over the whole surface, caps included.

Where 2D and 3D numbers must not be pooled

- **Mean island separation** — 2D includes the self-distance, deflating it by $\kappa/(\kappa + 1)$; (1.12) excludes it.
- **Divergence** — 2D is the divergence of a cumulative *unit-vector* field at the last frame pair (μm^{-1}); 3D is a window-mean $\nabla \cdot \mathbf{v}$ (s^{-1}).
- **Curl** — 2D is a signed scalar (opposite vortices cancel); 3D is an unsigned magnitude (they add).
- **Area / Volume Quantity** — the 2D "Area Quantity" is $\mu\text{m} \cdot \text{px}$; the 3D Volume Quantity (0.7) is a true μm^3 .

Measurement caveats

- **Flow speeds are a lower bound** (gain 0.55–0.92 by feature scale); relative comparisons within a dataset are sound, absolute values are not (§3.3).
- **flow_downsample** is a fixed dataset-wide choice, not a performance dial — it changes Curl, Divergence and ξ_v by factors of 0.4–2.3 (§3.1).
- **The velocity correlation length ignores the reliability filter and the mask**, unlike every other flow metric (§3.2).
- **mesh_maxrad** must be scaled to the object — the default of 5 loses roughly half a thin object's volume; Mesh Volume Ratio (§4.8) is the detector.
- **Solidity sits ~1.5% above MATLAB regionprops3** by choice of hull convention (§4.9).
- **Meshing is not bit-reproducible across processes** (§4.1).